

HRMS

HR Management System

Void Labs

Group 06

Team Members



234168H

J.T.R.Ramanayaka



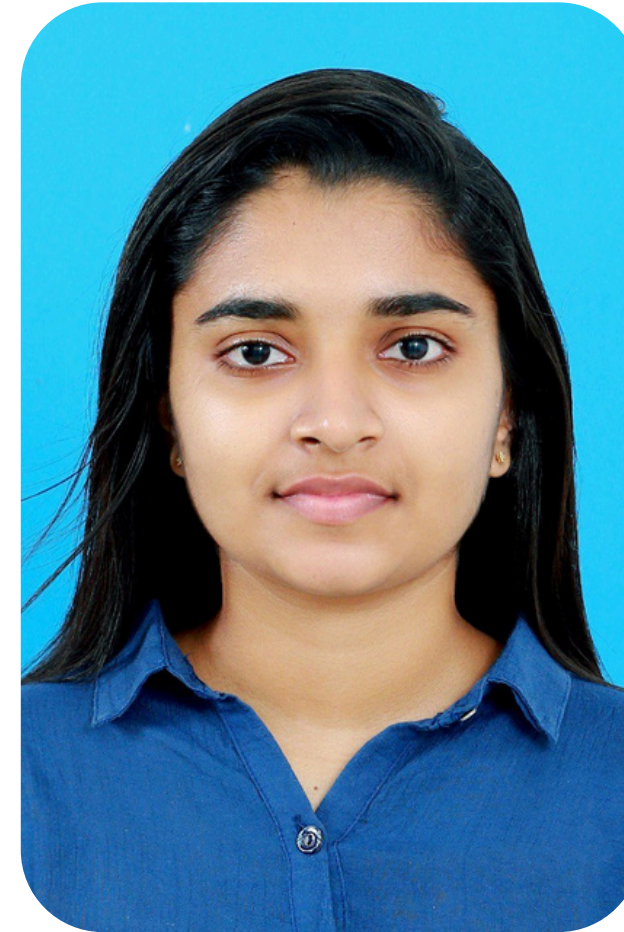
234054F

N.L.S.Dilshan



234185G

K.D.N.D.Samarakkodi



234145K

P.L.S.Nisansala



234028F

D.M.P.M.Bandara

Overview

- 01 Introduction
- 02 Problem in Brief
- 03 Aim and Objectives
- 04 Proposed Solution
- 05 Functionalities
- 06 Technologies
- 07 Software Process Model
- 08 Project Management Tool
- 09 Version control
- 10 Analysis and Design
- 11 User Interface
- 12 Implementation
- 13 Time Plan
- 14 Individual Contribution

Introduction

Overview

- Design and analysis of Insharp HRMS, a web-based Human Resource Management System
- Centralizes HR operations currently handled via manual and fragmented tools

Key Features

- Employee data, leave, recruitment, and document management
- Role-based access for employees, managers, and HR
- Automated workflows with real-time visibility and notifications
- Structured recruitment support with intelligent assistance

Purpose

- Reduce manual effort and administrative delays
- Improve transparency, accuracy, and efficiency in HR processes
- Provide a scalable and extensible HR system aligned with internal workflows



Problem in Brief

Fragmented HR processes spread across manual methods and multiple tools, causing delays, data inconsistency, and poor visibility.

High dependency on HR staff for routine tasks due to lack of employee self-service and real-time status tracking.

Inefficient and rigid recruitment & workflows, with manual screening, inconsistent evaluations, and limited system adaptability to changing policies.

Aim and Objectives

Aim

The primary aim of this project is to analyze, design, and plan a secure, scalable, and user-centric Human Resource Management System that reduces manual HR work, improves transparency, and aligns HR workflows with organizational policies. The proposed system serves as a centralized platform for core HR functions and provides a strong architectural foundation for future implementation.

Aim and Objectives

Objectives

- Analyze existing HR workflows to identify inefficiencies and automation opportunities
- Gather functional and non-functional requirements through stakeholder engagement
- Design a modular, role-based system architecture for key HR functions
- Define structured workflows aligned with approval hierarchies and policies
- Design secure role-based access control for sensitive HR data
- Plan AI-assisted recruitment support while retaining human decision authority
- Adopt an iterative Scrum-based development approach
- Prepare a detailed project plan covering scope, phases, and future implementation

Proposed Solution

Design a centralized, web-based HRMS tailored to organizational workflows, planned for incremental implementation beyond the interim stage.

Implement a role-based access model to ensure secure and relevant interaction for employees, managers, HR officers, and administrators.

Support core HR functions (employee data, recruitment, leave, documents, notifications, reporting) using modular and workflow-driven design to improve transparency and consistency.

Integrate AI-assisted recruitment support as a decision-support tool, while delivering detailed requirements, architecture, and planning at the interim stage.



Functionalities

Employee & HR Operations

- Secure user login and authentication
 - Role-based access for employees and HR staff
 - Employee profile and information management
 - Leave application, balance tracking, and leave history
 - Document and official letter management
 - System notifications for HR activities
 - Internal messaging for targeted communication
- 

Management & System Support

- Recruitment and vacancy management
- CV upload and candidate profile handling
- Interview scheduling and evaluation management
- AI-assisted recruitment support for candidate screening
- Workflow-based approvals aligned with policies
- HR reports and analytics generation
- Secure handling of sensitive HR data

Technologies

Front end

- Next.js v16 (LTS) – Modern, responsive web UI with server-side rendering and optimized performance
- Web Browsers – Chrome, Firefox, Safari, Edge (latest versions)
- Responsive Design – Supports desktop and mobile screens



Backend

- FastAPI (Python 3.11) – High-performance REST API framework for business logic and workflows
- Modular Monolithic Architecture – Logically separated HR modules within a single backend
- AI Service (FastAPI) – Separate internal service for AI-assisted CV processing



Database

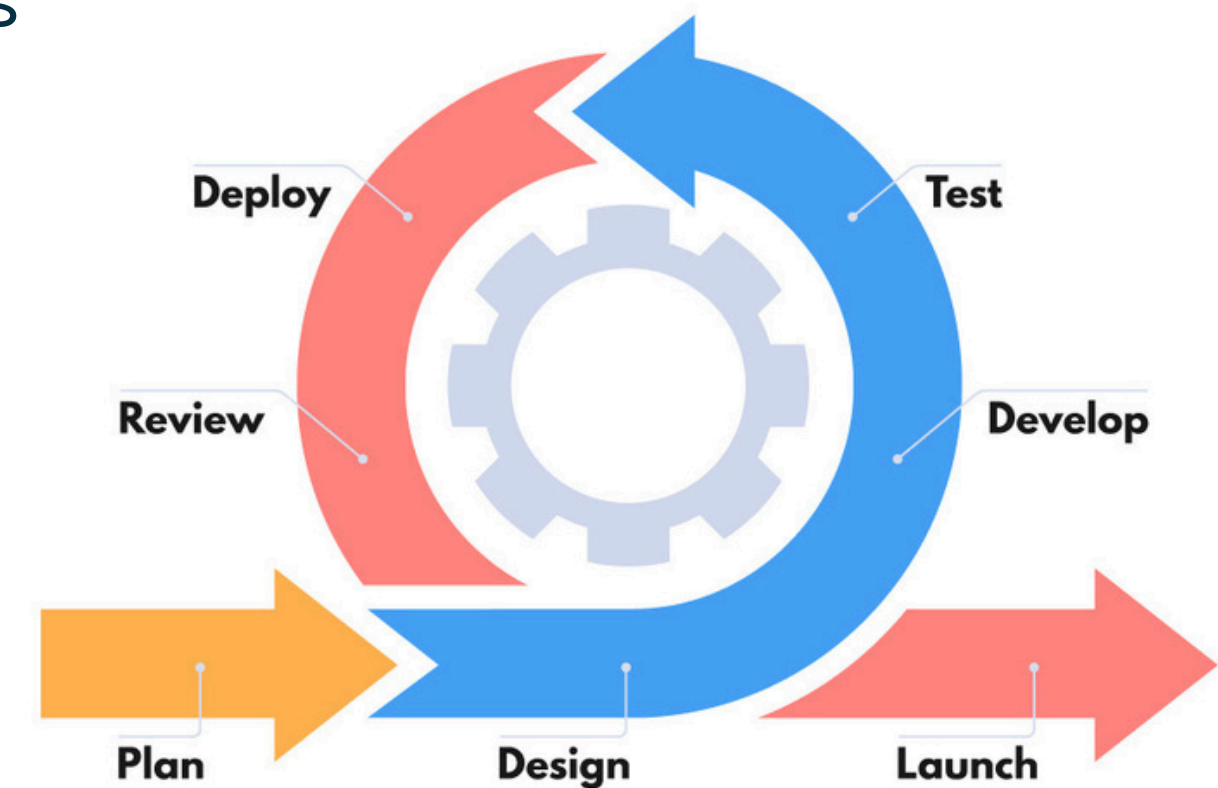
- PostgreSQL v15 – Centralized relational database with strong data integrity
- SQLAlchemy ORM – Database abstraction and transaction management
- S3-Compatible Object Storage – Secure storage for documents and files



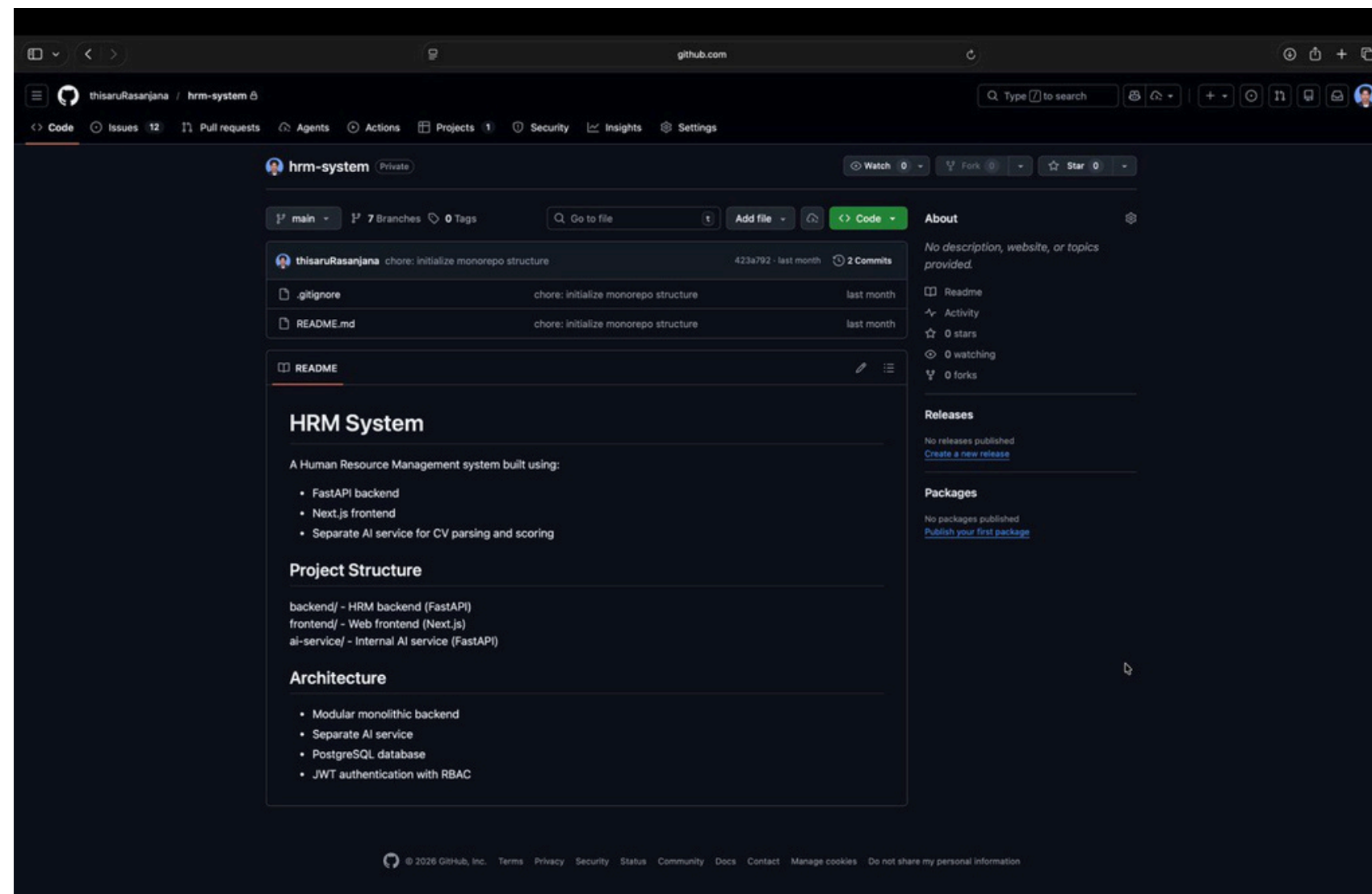
Software Process Model

We adopt the Agile Scrum process model to develop the HRMS in short, iterative sprints. Agile allows us to deliver functional components incrementally, gather continuous stakeholder feedback, and adapt quickly to evolving HR requirements. This approach is well-suited for the project's modular design and supports effective planning, collaboration, and risk reduction during development

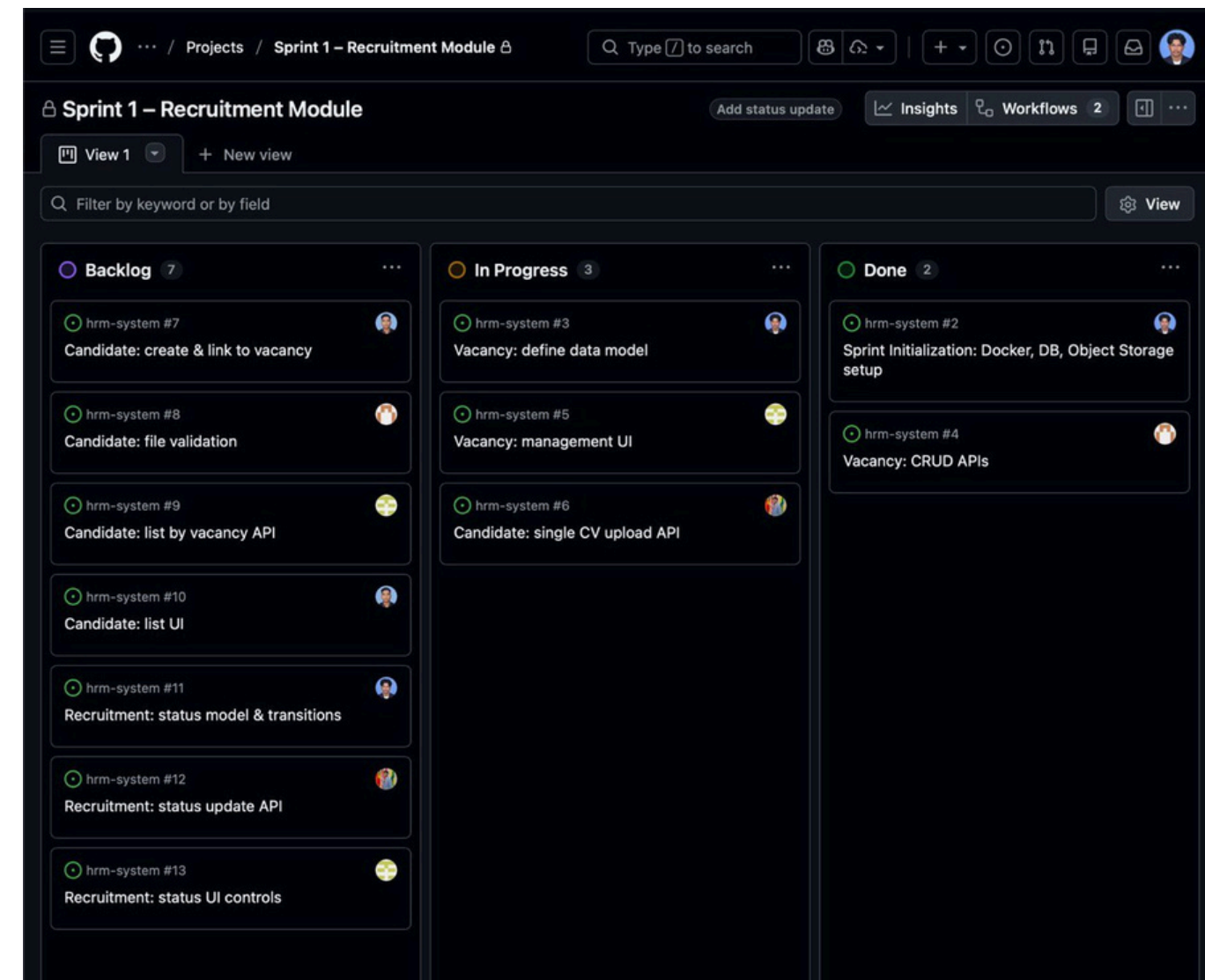
SCRUM METHODOLOGY



Version Controlling and Project Management



Git



Github Projects

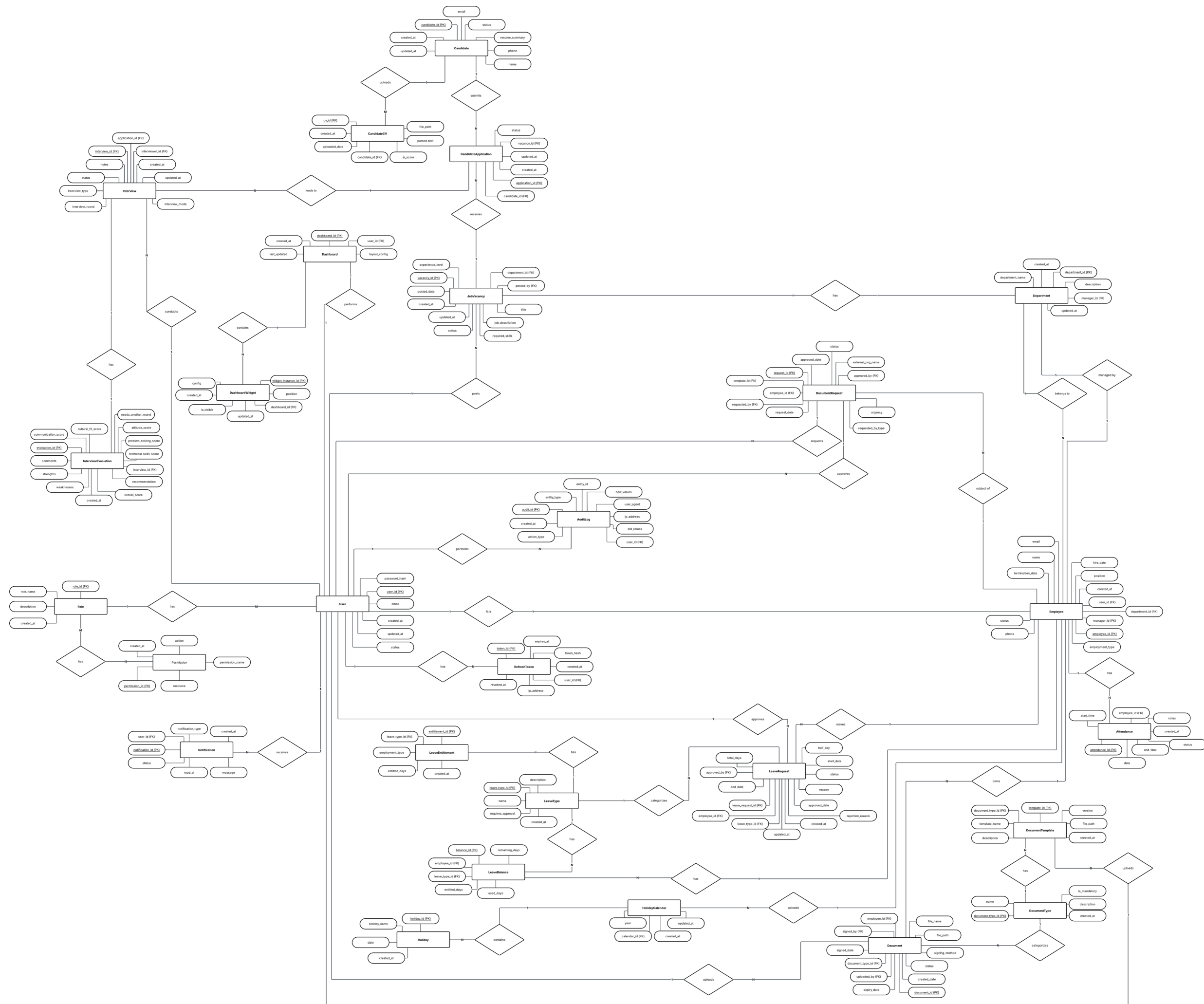
Analysis and Design

Diagrams

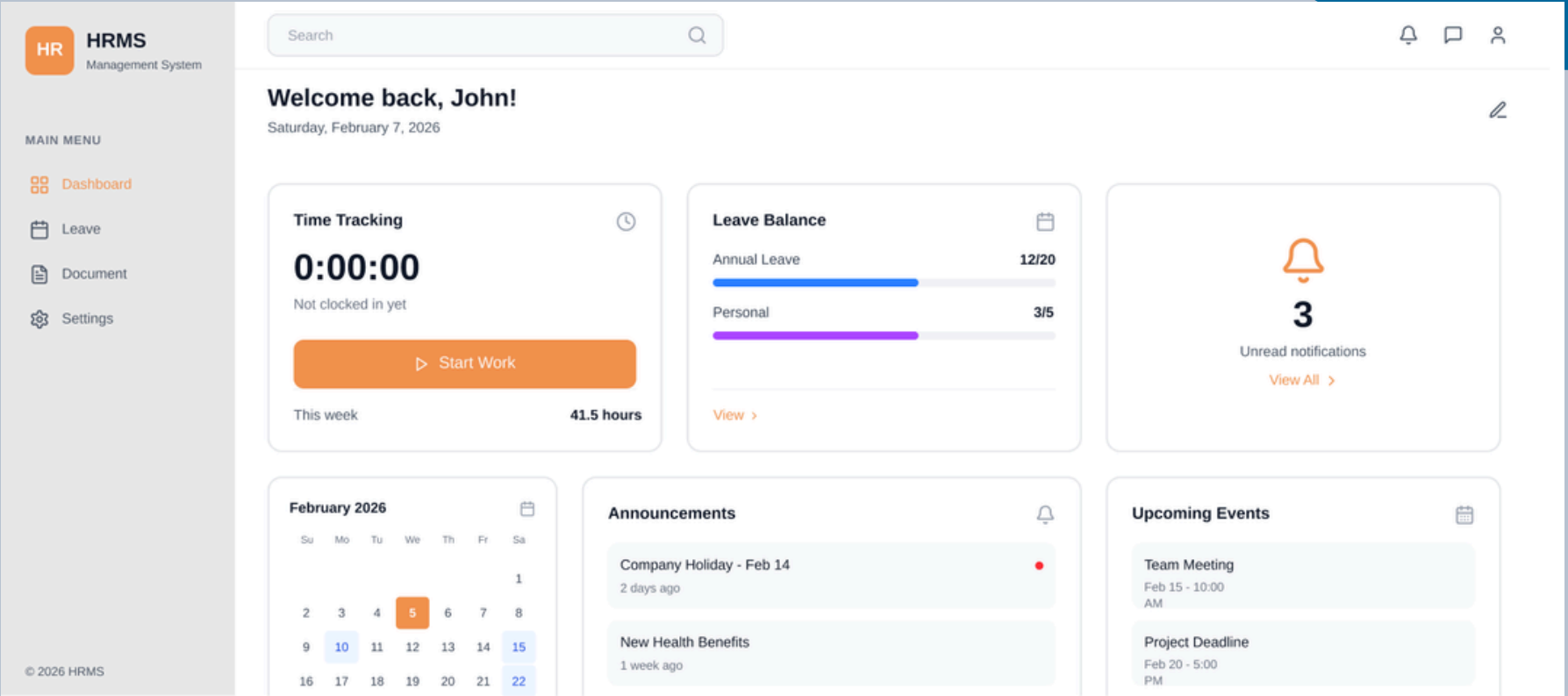
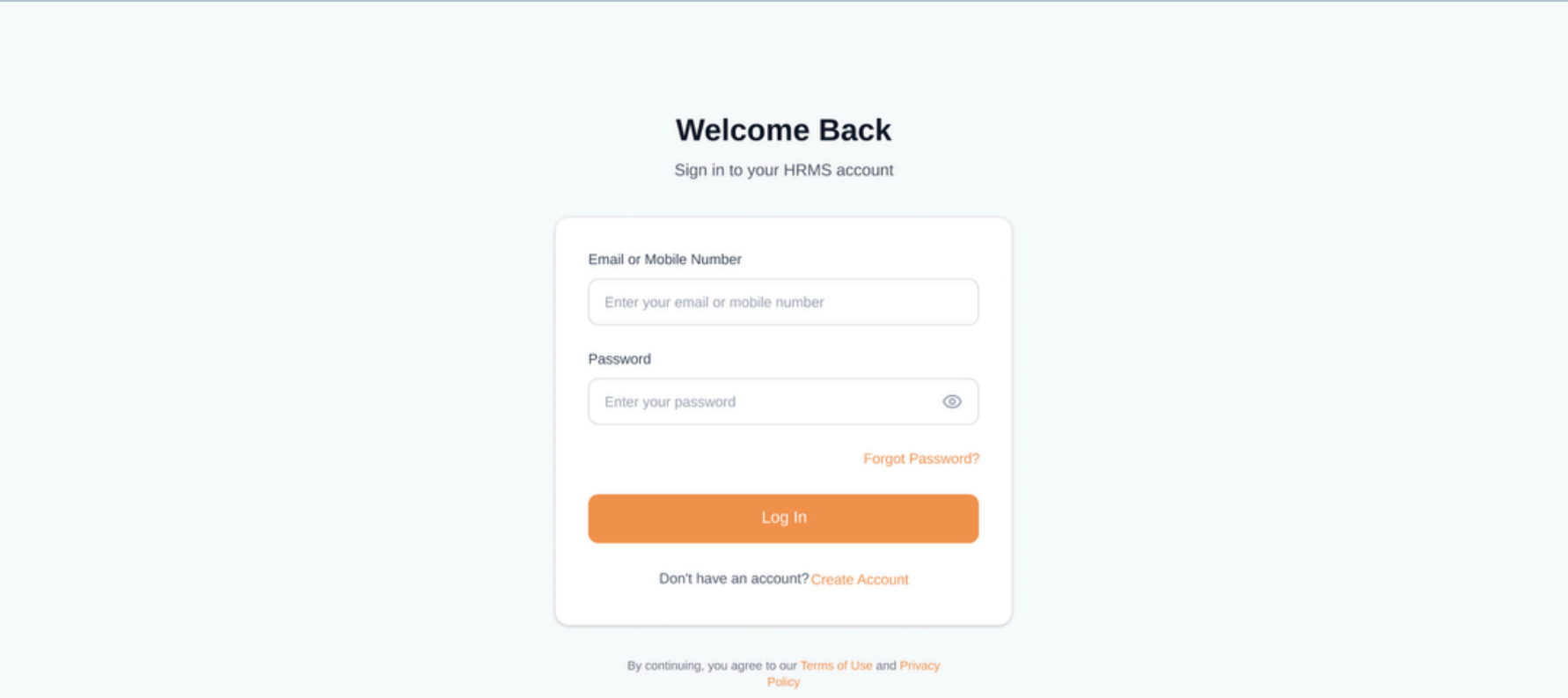
- Use case Diagram
- ER Diagram
- Class Diagram
- Activity Diagram
- Sequence Diagram

[Lucid chart link](#)

ER Diagram



UI Designs



[Figma Design link](#)

Implementation

EXPLORER

HRM-SYST...
▼ backend
 ▼ app
 ▼ database
 > documents
 > employees
 > leave
 ▼ recruitment
 > __pycache__
 > __init__.py
 > models.py
 > router.py
 > schemas.py
 > service.py
 > __init__.py
 > main.py
 > .env
 > .env.example
 > requirements.txt
 ▼ frontend
 > .next
 ▼ app
 ▼ recruitment
 ▼ create
 > page.tsx
 > page.tsx
 > favicon.ico
 > # globals.css
 > layout.tsx
 > page.tsx
 > node_modules
 > public
 > .gitignore
 > JS eslint.config.mjs
 > TS next-env.d.ts
 > TS next.config.ts
 > {} package-lock.json
 > {} package.json
 > OUTLINE
 > TIMELINE
 > PRETTY TYPESCRIPT ERROR

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49

frontend > app > recruitment > create > page.tsx > VacancyCreatePage
1 "use client";
2
3 import { useState } from "react";
4 import { useRouter } from "next/navigation";
5
6 export default function VacancyCreatePage() {
7 const router = useRouter();
8
9 const [form, setForm] = useState({
10 title: "",
11 department: "",
12 experience_level: "",
13 description: "",
14 });
15
16 const handleSubmit = async (e: React.FormEvent) => {
17 e.preventDefault();
18
19 await fetch("http://127.0.0.1:8000/recruitment/vacancies", {
20 method: "POST",
21 headers: { "Content-Type": "application/json" },
22 body: JSON.stringify(form),
23 });
24
25 router.push("/recruitment");
26 };
27
28 return (
29 <div className="flex min-h-screen bg-gray-100">
30 <div>/* SIDEBAR */</div>
31 <div>
32 <h1>HRSM</h1>
33
34 <p>MAIN MENU</p>
35
36 <nav>
37 <div>Dashboard</div>
38 <div>Recruitment</div>
39 <div>Document</div>
40 </nav>
41 </div>
42 <div>/* MAIN CONTENT */</div>
43 </div>
44);
45
46 </div>
47
48 </div>
49

EXPLORER

HRM-SYSTEM
▼ backend
 ▼ app
 ▼ database
 > documents
 > employees
 > leave
 ▼ recruitment
 > __pycache__
 > __init__.py
 > models.py
 > router.py
 > schemas.py
 > service.py
 > __init__.py
 > main.py
 > .env
 > .env.example
 > requirements.txt
 ▼ frontend
 > .next
 ▼ app
 ▼ recruitment
 ▼ create
 > page.tsx
 > page.tsx
 > favicon.ico
 > # globals.css
 > layout.tsx
 > page.tsx
 > node_modules
 > public
 > .gitignore
 > JS eslint.config.mjs
 > TS next-env.d.ts
 > TS next.config.ts
 > {} package-lock.json
 > {} package.json
 > OUTLINE
 > TIMELINE
 > PRETTY TYPESCRIPT ERROR

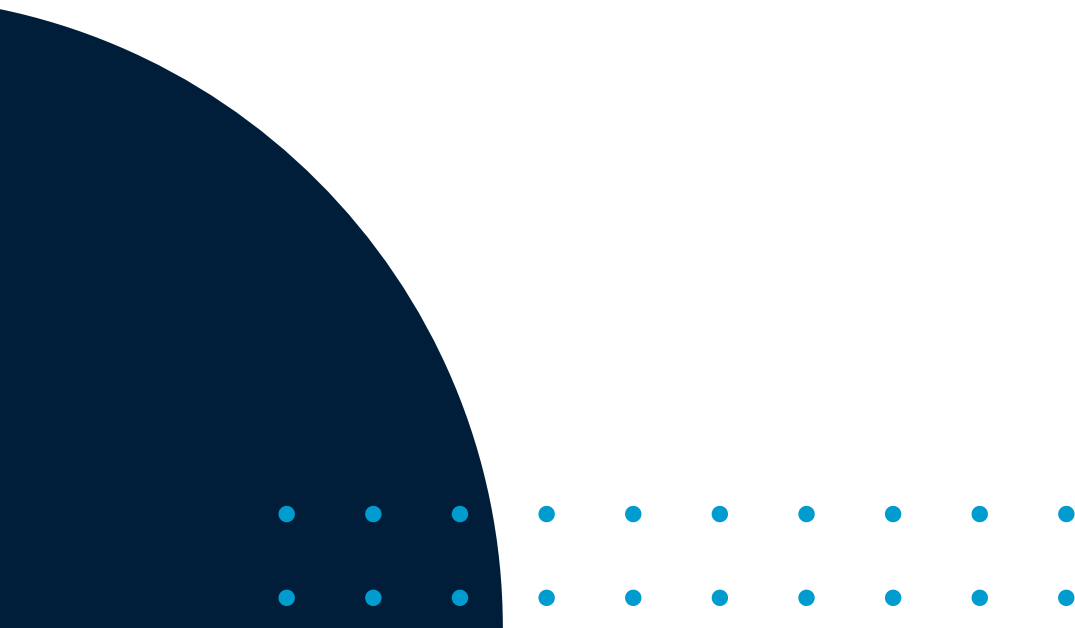
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35

backend > app > main.py > startup
1 from fastapi import FastAPI
2 from fastapi.middleware.cors import CORSMiddleware
3
4 from app.database.database import engine
5 from app.database.base import Base
6 from app.recruitment import models as recruitment_models
7 from app.recruitment.router import router as recruitment_router
8
9 app = FastAPI(title="HRM Backend")
10
11 app.add_middleware(
12 CORSMiddleware,
13 allow_origins=[
14 "http://localhost:3000",
15 "http://127.0.0.1:3000",
16],
17 allow_credentials=True,
18 allow_methods=["*"],
19 allow_headers=["*"],
20)
21
22 @app.on_event("startup")
23 def startup():
24 try:
25 with engine.connect() as connection:
26 print("Database connected successfully")
27 except Exception as e:
28 print("WARNING: Database connection failed")
29 print(e)
30
31 app.include_router(recruitment_router)
32
33 @app.get("/")
34 def root():
35 return {"message": "HRM backend with DB connected"}

17



Individual Contribution



Requirements Analysis

- Identified requirements for Authentication, Dashboard, Messaging & Notifications based on user roles and usability needs.

System Design

- Designed ER, Class, Use Case, Activity & Sequence diagrams for Authentication, Dashboard, Messaging & Notification modules.

UI Design

- Created interfaces for Login & Authentication, Customizable Dashboard, Notification Center & Internal Messaging.

Core Features

- Designed JWT-based authentication, dashboard data aggregation, internal messaging & system-generated notifications.

Project Planning

- Contributed to implementation planning aligned with Scrum.

Requirements Analysis

- Defined functional and non-functional requirements for Employee Management and Role-Based Access Control (RBAC).

System Design

- Designed ER, Class, Use Case, Activity & Sequence diagrams for Employee Management & RBAC modules.

UI Design

- Created interfaces for Employee Management and Assign Role & Permissions

Employee Management & RBAC

- Designed employee CRUD operations, role-permission structure, and backend API-level access enforcement.

Project Planning

- Contributed to architectural planning and task alignment with Scrum.

J.T.R. Ramanayaka

234168H

Requirements Analysis

- Identified recruitment workflow issues and defined key functional requirements.

System Design

- Designed ER, Class, Use Case, Activity & Sequence diagrams for recruitment module.

UI Design

- Created interfaces for vacancies, CV upload, AI-ranked candidates, interviews & evaluations.

AI CV Filtering

- Designed CV parsing, skill matching, scoring logic & ethical AI integration.

Project Planning

- Contributed to task planning aligned with Scrum.

Requirements Analysis

- Identified issues in leave, attendance, and reporting processes; defined related functional requirements.

System Design

- Designed ER, Class, Use Case, Activity & Sequence diagrams for Leave Management, Attendance & Reporting modules.

UI Design

- Created interfaces for leave requests, approvals, attendance marking, dashboards & report export.

Leave, Attendance & Reporting

- Designed leave balance logic, approval workflows, attendance tracking & PDF/CSV report generation.

Project Planning

- Contributed to task planning aligned with Scrum.

Requirements Analysis

- Identified issues and defined requirements for Document Management and Letter Generation.

System Design

- Designed ER, Class, Use Case, Activity & Sequence diagrams for the Document module.

UI Design

- Created interfaces for document upload, approval workflows & letter generation.

Document Management and Letter Generation.

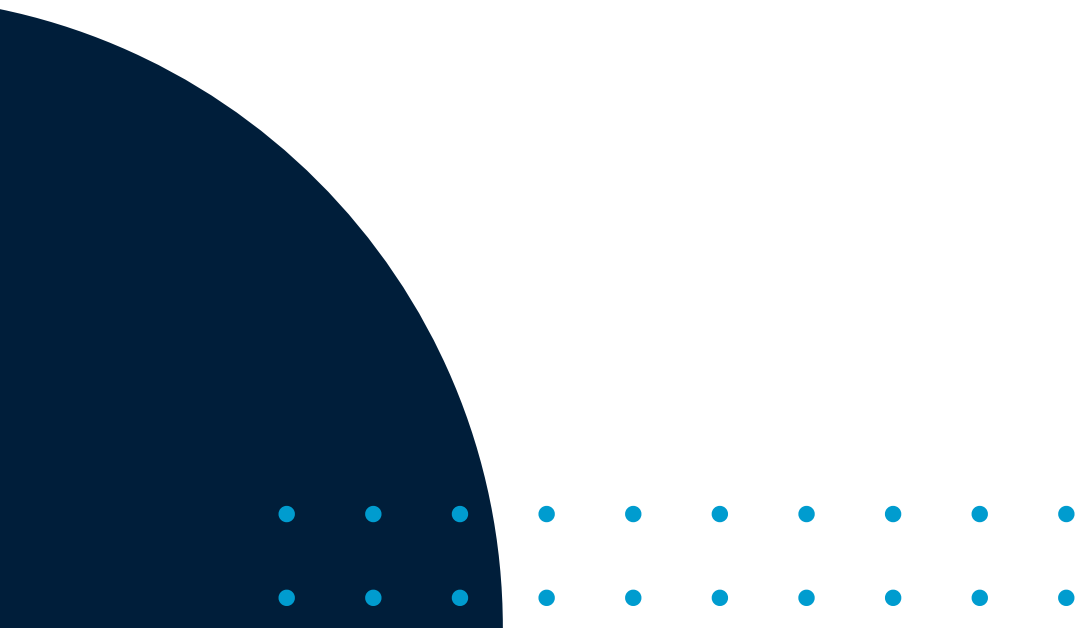
- Designed secure storage, role-based access & template-based auto-filled letter generation.

Project Planning

- Contributed to task planning aligned with Scrum.



Thank you!



Q & A